

Machine learning text classifiers and open and closed-source large language models (LLMs) in political science

Day 1

Joshua Cova and Luuk Schmitz

2026-04-23

Overview

Part 1: Vectorization and word embeddings

Part 2: Transformers, transfer learning and encoder-only models

Part 3: BERT models and the Hugging Face interface

Part 1: Vectorization and Word Embeddings

Where we left things last time

Hopefully (some of) this will still be familiar to you:

- Bag-of-words (BoW) models
- Classification metrics
- Training and testing
- Naive Bayes
- Supervised vs. Unsupervised machine learning

But in case, it is not...

Bag-of-words models

- Bag-of-word model: a theoretical idea of how one can represent text computationally
- The data representation is called the **Document-Term Matrix (DTM)** with documents and terms as rows and columns.
- Represent text numerically (text tokenization)
- Ignores grammar and word order
- Vocabulary built from unique words

Classification metrics

- Accuracy: proportion of correct predictions
- Precision: correctness of positive predictions
- Recall: ability to find all positive cases
- F1-score: balance between precision and recall

Training and testing

- Identify a sample from your corpus
- Split data into **training** and **test** sets
- Training set used to learn patterns
- Test set used to evaluate performance
- Remember that over-fitting could be a problem...

How can we find out how well we did?

- Last time, we looked at Naive Bayes
- Probabilistic classification algorithm, based on Bayes' theorem (spam classification exercise)
- Assuming feature independence

Supervised vs. Unsupervised Machine Learning

- **Supervised:** Uses labeled data
- **Unsupervised:** Finds *latent* patterns in unlabeled data
- Examples:
 - Supervised: Spam detection, text classification
 - Unsupervised: Identifying themes in large corpora

Connecting to what you already know

The units of analysis so far have been: **word counts** and the **Term Frequency - Inverse Document Frequency (TF-IDF)** metric.

We have identified some limitations of this approach:

- **Polysemy:** “bank” (finance) = “bank” (river)
- **Synonymy:** “happy” and “joyful” are completely unrelated features
- **Context blindness:** word order and relationships are ignored
- **Compositionality problem:** meaning often comes from word combination, e.g. “white house” $\neq$ “white”, “house”
- **Sparse matrices:** most entries in the matrices are 0.

Today we ask: what if words were **points in geometric space**, and meaning was **distance**?
Sounds abstract, let's explore this further!

Can we improve on the previous representation?

What we want from a word embedding $\rightarrow$ A numerical representation of meaning

Why is this important? BoW models count frequencies, without “understanding” the meaning.

Our *desiderata*:

1. Similar words should be close together

- “good” near “excellent”, should be far from “terrible”
- “democracy” should be close to “election”, far away from “fruit”

2. Relationships should be preserved geometrically

- king : queen :: man : woman

3. Context should eventually shape meaning

- “bank” in a finance article $\neq$ “bank” beside a river $\rightarrow$ Contextual embeddings

4. This is what **word embeddings** provide. Concretely we want to transform words into dense matrices (i.e. a collection of numbers).

The distributional hypothesis

“You shall know a word by the company it keeps.” — J.R. Firth (1957)

Words that appear in **similar contexts** tend to have **similar meanings**.

- “great” and “awesome” often appear after “this movie is” → similar vectors
- “cat” and “dog” often appear near “feed”, “pet”, “vet” → similar vectors
- “cat” and “democracy” almost never share contexts → distant vectors

Application from political science:

“If the distance between “immigrants” and “hardworking” is smaller for liberals than for conservatives, we learn something about their relative worldviews” (Rodriguez and Spirling, p.101)

A little philosophical detour

“The meaning of a word is its use in the language” — Ludwig Wittgenstein (1953)

- Meaning emerges from how words are used in different contexts
- Words that appear in similar contexts tend to have similar meanings.
- Meaning is inferred from real usage patterns, not from dictionary definitions
- Idea of “family resemblance” of words (*Familienähnlichkeit*).
- Think of the concept of “game”. There is no single feature shared by all games.
- Concepts are connected by overlapping similarities, not strict definitions.

GloVe and Word2Vec: Learning embeddings from context

GloVe (Pennington, Socher & Manning, 2014) and *Word2Vec* (Mikolov et al., 2013) introduced two widely used word embedding models:

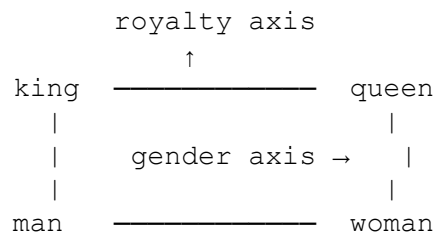
- **CBOW** (Continuous Bag of Words): Predict the **target word** from surrounding context words (sliding window)
- **Skip-gram**: Predict **context words** from the target word
- **Co-occurrence matrix**: $X_{ij} = \text{frequency of word } j \text{ in context of } i$

The **size** of the embedding also matters: we want semantic representation and the right dimensionality (trade-off between lower vs. higher dimensions).

Words that **predict similar contexts** end up with **similar weight vectors**.

The geometry of meaning

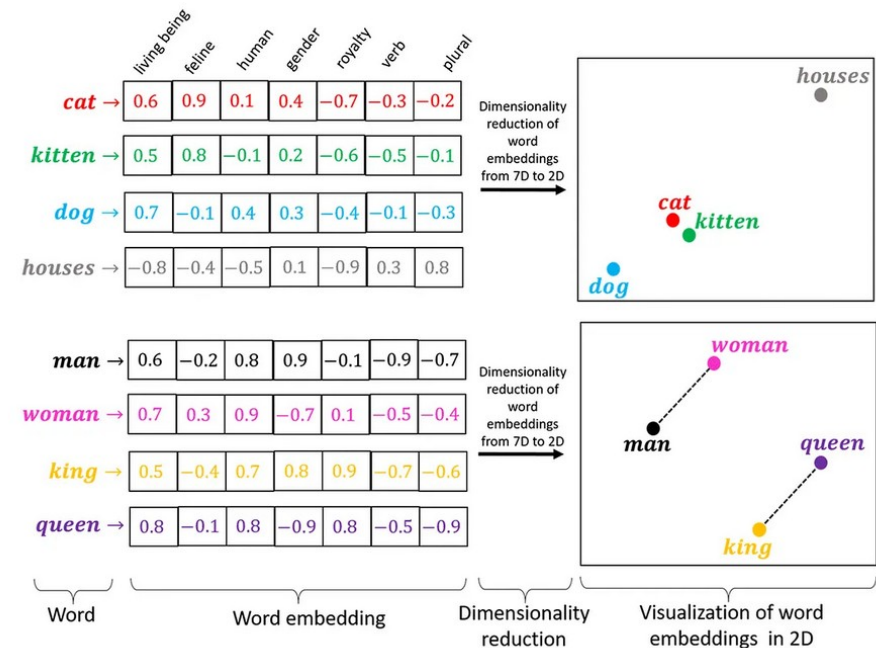
Semantic relationships become geometric relationships.



$$\begin{aligned} & \left[\vec{\text{king}} - \vec{\text{man}} + \right. \\ & \left. \vec{\text{woman}} \right] \approx \\ & \left[\vec{\text{queen}} \right] \end{aligned}$$

The model was never told anything about gender or royalty — it emerged **inductively** from co-occurrence.

Different semantic connections are leveraged (e.g. gender, tense, grammar), but existing biases in training data can be replicated



But what do these dimensions actually represent?

Let's see how this works in practice: Code + Maths

Vectors — A word becomes a list of numbers

```
king   = [ 0.7, 0.8, 0.9]
queen  = [ 0.8, -0.9, 0.8]
man     = [ 0.8, 0.9, -0.1]
woman  = [ 0.9, -0.7, 0.1]
```

Vector Addition & subtraction

Vector magnitude

$$\|\mathbf{A}\| = \sqrt{a_1^2 + a_2^2 + \dots + a_n^2}$$

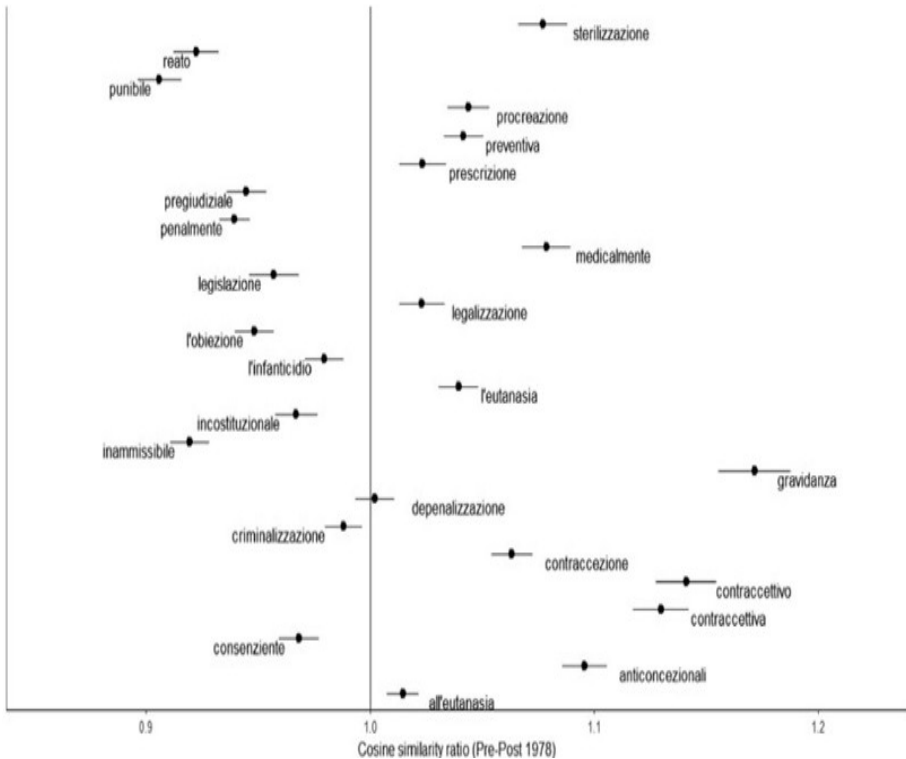
Dot product — multiply pairs, then sum

$$\mathbf{A} \cdot \mathbf{B} = \sum_{i=1}^n a_i b_i$$

Cosine similarity — dot product normalised by length

$$\text{sim}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \times \|\mathbf{B}\|}$$

Word embeddings in action (political science)



Cova (2025)

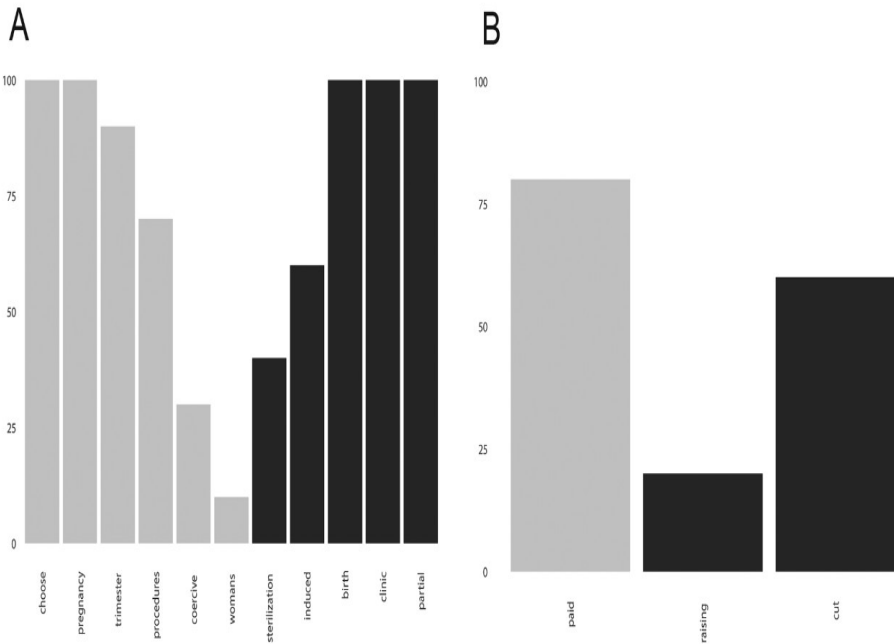


Figure 7. Nearest neighbors most common to each party. Democrat = light gray; Republican = dark gray. A, Abortion. B, Taxes

Spirling and Rodriguez (2022)

Word embeddings in action (sociology)

- The historical trajectory of gender bias (Jones et al. 2020)

Using a corpus of millions of digitized books from 1800s to 2000 Jones et al. (2020) calculated the mean cosine distance between lists of gendered words (e.g. “man”, “woman”) and domain-specific words (e.g. “career”, “family”). Although gender association have decreased in time they remain ‘sticky’

- Intersectionality in mental health (Johnson, 2021)

Use Word2Vec to examine how mental health terminology is racialized and gendered. Measured via cosine similarity how the concepts of “illness” and “criminality” cut across racial and gender lines.

- But from a sociological perspective, what do word embeddings actually measure? **Context vs. Concept spaces** (*Meaning in Hyperspace: Word Embeddings as Tools for Cultural Measurement* (Boutyline and Arseniev-Koehler, 2025))

The problem with static embeddings

Static word embeddings: every word has **one** fixed vector.

- “She deposited money at the **bank**”
- “The fisherman sat on the river **bank**”
- “The plane made a sharp **bank** to the left”

All three “bank”s get the **same vector** — an average over all its meanings.

The representation we actually want:

- The *bank* in sentence 1 should be close to *money, finance, account*
- The *bank* in sentence 2 should be close to *river, shore*
- The *bank* in sentence 3 should be close to *turn, tilt*

This requires **contextual embeddings** — which brings us to Part 2.

Part 2: Transformers, Transfer Learning and Encoder-Only Models

Can we improve on the previous representation?

Bag-of-words models

- Words as count
- No order/meaning

Static word embeddings

- Words as vectors
- Words assume a fixed meaning, but no context

Contextual word embeddings

- Words in context
- Let words define each other dynamically

Key intuition: Meaning is not stored, it is constructed

The transformer revolution

Vaswani et al. (2017): “Attention is All You Need”

The conceptual jump: Pre-Transformers, word embeddings were a (very good) look-up table of word vectors.

Innovation: Thanks to Transformers, we have language models that can compute contextual embeddings on the fly.

Transformers process **all words in parallel** and use **attention** to relate any word to any other word, regardless of distance.

A synthesis of some core ideas/innovations: **attention**, **transfer learning** and **neural networks**

The encoder-decoder framework

Classical NLP framing for sequence-to-sequence tasks (e.g. translation):

Input: "Transformers are great!" (English)

↓

[ENCODER]

↓

Hidden state vector ← compressed representation of the input

↓

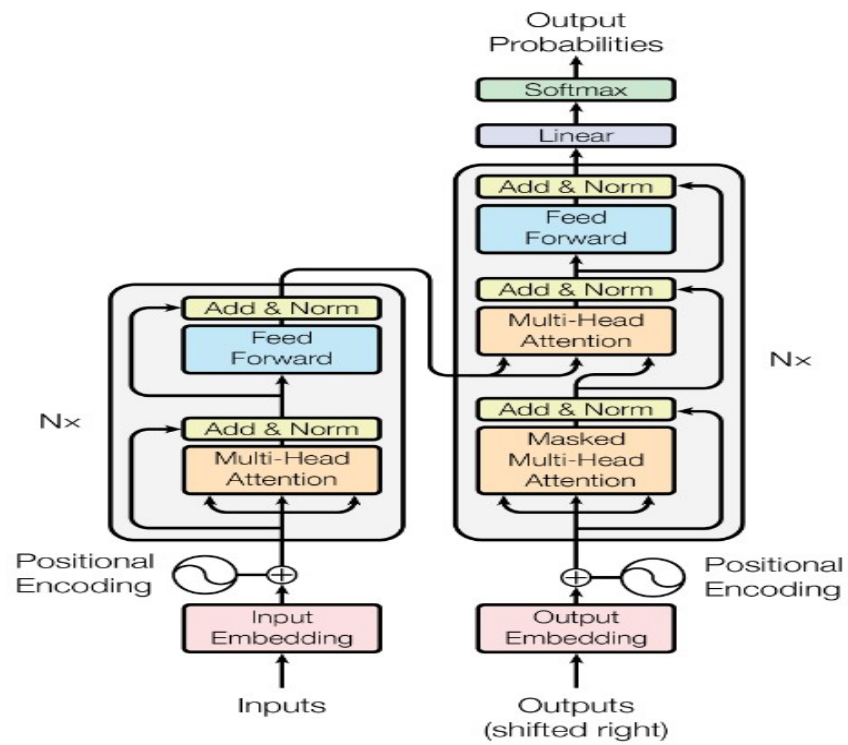
[DECODER]

↓

Output: "Transformer sind toll!" (German)

The encoder **reads** and **understands**. The decoder **generates**.

Transformers: the visualization



Transformers: Step-by-step

1. **Tokenize** the text
2. Convert tokenized text into vectors: **static embedding + positional encoding**
3. **Self-attention**: Each word looks at every other word and decides: “How important are they for understanding me?”
 - “The movie was surprisingly good” vs. “good morning”, context-aware embeddings
 - This is done computing the Query, Key and Value of every word (weight matrices)
4. **Multi-head attention**: Instead of one attention process, the model runs several in parallel (one “might” focus grammar, sentiment, relationship etc..).
 - Why “might”?
5. **Encoding Layer**: After attention, each token goes through a neural network. The process is repeated several times. Multiple layers refine the meaning

Transformers: The output

The original translation task: Each encoder layer then goes through a decoder, which generates the output step by step (token by token).

Classification: A summary [CLS] vector summarizes the information, which then goes through a final layer (Softmax algorithms converts logits into probabilities) $\rightarrow$ Label prediction

Self-attention: Words talking to each other

In transformers, **encoder layers** use **self-attention**: every word attends to every other word in the same sequence.

Example: “The pizza came out of the oven and it tasted good.”

When encoding “**it**”, the model attends across all words:

Word	Attention weight
pizza	0.72
oven	0.18
came	0.05
...	...

“It” gets updated toward “pizza” — the model learns the referent.

Transfer learning: Pre-train once, fine-tune many times

Traditional supervised learning:

- Collect labelled data for your task
- Train a model from scratch
- Need thousands or millions of examples
- Repeat for every new task

Transfer learning:

- Pre-train on massive unlabelled corpus (self-supervised)
- Produce a general-purpose language model
- Fine-tune on small labelled dataset for your task
- The pre-trained weights encode general linguistic knowledge

This is the paradigm shift that made NLP accessible to social scientists.

Encoder-only vs. decoder-only models

	Encoder-only	Decoder-only
Examples	BERT, RoBERTa, DistilBERT	GPT-2, GPT-4, LLaMA
Attention	Bidirectional (sees full context)	Causal (sees only past)
Pre-training	Masked Language Modelling	Next-token prediction
Good for	Classification, NER, similarity	Generation, completion, chat
Day 1 focus	✓	(Day 2)

For **understanding** tasks in social science → encoder-only.

For **generation** tasks (annotation via prompting, interviews) → decoder-only.

Part 3: BERT Models and the Hugging Face Interface

BERT: Bidirectional Encoder Representations from Transformers

Devlin et al. (2018) — Google AI

Transformer is the architecture; BERT is a specific model built using that architecture

BERT applies the transformer **encoder** with a novel pre-training scheme on:

- 3.3B words from BookCorpus + English Wikipedia

BERT's pre-training: Masked Language Modelling

Masked Language Modelling (MLM):

Input: "The [MASK] came out of the oven and it tasted [MASK]."
Target: "pizza" "good"

- Randomly mask 15% of input tokens
- Train the model to predict the masked words
- Requires understanding **both left and right context**

Fine-tuning BERT for your task

- Even with **a few hundred examples**, fine-tuned BERT often outperforms fully trained BoW classifiers (Wang, 2023; Timoneda and Vera, 2025)
- Explosion of domain-specific BERT models: General BERT was trained on Wikipedia and books. For specialized domains, domain-adapted variants often perform better (FinBERT, CamemBERT)
- The Hugging Face ecosystem: sharing and using pre-trained NLP models

Practical considerations

Issue	Guidance
Token limit	BERT handles max 512 tokens; longer texts need chunking or a long-context model
Compute	GPU strongly recommended for fine-tuning; inference on CPU is slow but feasible
Reproducibility	Set random seeds; log model name and version
Interpretability	Embeddings are black-box; validate outputs against human annotations

What's coming on Day 2

Decoder-only models and LLMs for text analysis

- How do LLMs work?
- How should researchers interact with LLMs in a scientific setting?
- Open vs. closed-source alternatives
- Prompt engineering for zero-shot classification
- Other use cases: simulations, interviews, metadata extraction
- Quantization and locally hosted alternatives
- Group exercise: Using LLMs locally for text classification
- Brainstorming: How does this apply to your own research?

Thank you!

Questions?

Tomorrow: From understanding to generation — how LLMs work and how to use them responsibly in research.

References

- Boutyline, A. and Arseniev-Koehler, A., 2025. Meaning in hyperspace: Word embeddings as tools for cultural measurement. *Annual review of sociology*, 51.
- Cova, J., 2025. A new database for Italian parliamentary speeches: introducing the ItaParlCorpus dataset. *Italian Political Science Review/Rivista Italiana di Scienza Politica*, 55(1), pp.77-86.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.
- Firth, J. R. (1957). A synopsis of linguistic theory 1930-55. *Studies in Linguistic Analysis*.
- Johnson, A.L., 2024. Psychotic white men and bipolar black women? Racialized and gendered implications of mental health terminology. *Social science & medicine*, 352, p.117015.
- Jones, J.J., Amin, M.R., Kim, J. and Skiena, S., 2020. Stereotypical gender associations in language have decreased over time. *Sociological Science*, 7, pp.1-35.

References

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.

Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. *EMNLP*.

Rodriguez, P.L. and Spirling, A., 2022. Word embeddings: What works, what doesn't, and how to tell the difference for applied research. *The Journal of Politics*, 84(1), pp.101-115.

Timoneda, J.C. and Vera, S.V., 2025. BERT, RoBERTa, or DeBERTa? Comparing performance across transformers models in political science text. *The Journal of Politics*, 87(1), pp.347-364.

Tunstall, L., von Werra, L., & Wolf, T. (2022). *Natural Language Processing with Transformers*. O'Reilly.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *NeurIPS*.